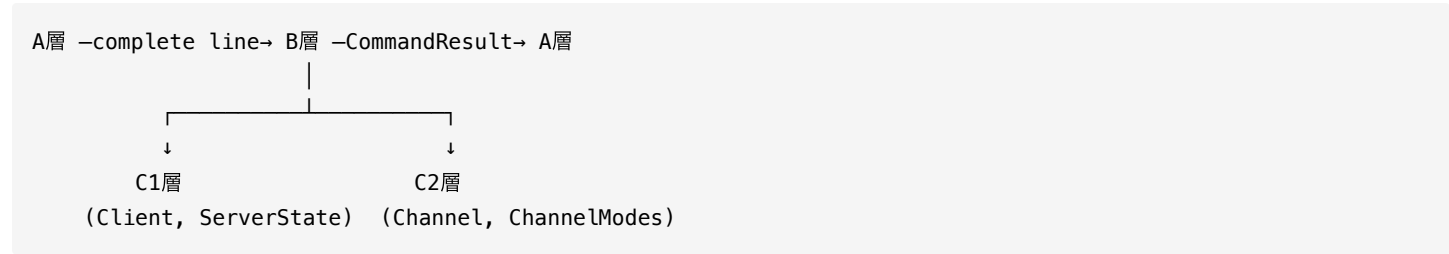


層間インターフェース仕様

本ドキュメントは、ircserv の層間API契約を定義する。
各層の責務は design.md を参照。

スコープ



境界	方向	内容
A→B	入力	complete line (<code>\r\n</code> 区切り文字列)
B→A	出力	<code>CommandResult</code> (送信先fd + 送信文字列)
B↔C1	双方向	<code>Client</code> , <code>ServerState</code> 操作API
B↔C2	双方向	<code>Channel</code> , <code>ChannelModes</code> 操作API

チーム構成

担当	メンバー	範囲
A	torinoue	Network / IO (Server, Connection)
B	torinoue	Protocol / Command (Parser, Dispatcher, ReplyBuilder)
C1	taro	Client / ServerState
C2	hanako	Channel / ChannelModes

1. 境界オブジェクト (A↔B)

1.1 CommandResult

B層からA層への戻り値。送信先fdと送信文字列を保持。

```
struct CommandResult {
    std::vector<OutgoingMessage> replies;
    bool shouldDisconnect;

    CommandResult();
    void addReply(int fd, const std::string& message);
};
```

役割:

- コマンド処理後に送るべきメッセージを保持
- 必要に応じて切断要求を表す
- BがAの `Connection` や `Server` を直接触らないための境界

1.2 OutgoingMessage

```
struct OutgoingMessage {
    int fd;
    std::string message;

    OutgoingMessage(int targetFd, const std::string& text);
};
```

役割:

- Bが「誰に何を送るか」を表現
- 実際の `send()` は行わない
- A / Server がこの情報を見てsend bufferへ積む

2. A層概要（Network / IO）

詳細は `onboarding_A.md` 参照

クラス	責務
<code>Server</code>	メインループ、イベント振り分け、 <code>CommandResult</code> 適用
<code>Connection</code>	fd、recv/send buffer、ノンブロッキングI/O
<code>Poller</code>	(optional) pollfd管理
<code>ConnectionManager</code>	(optional) Connection辞書管理

A層はIRCコマンドの意味を知らない。

3. B層概要（Protocol / Command）

詳細は `onboarding_B.md` 参照

クラス	責務
<code>Parser</code>	文字列 → <code>Message</code> 変換

クラス	責務
Message	IRCメッセージ構造体
CommandDispatcher	コマンド振り分け、状態更新、結果生成
ReplyBuilder	返信文字列生成（Numeric Reply等）

B層はNetwork/IOクラスに依存しない。送信は `CommandResult` 経由。

3.1 Message

```
class Message {
private:
    std::string          _command;
    std::vector<std::string> _params;
public:
    const std::string& getCommand() const;
    const std::vector<std::string>& getParams() const;
    size_t getParamCount() const;
    const std::string& getSingleParam(size_t index) const;
    bool hasParam(size_t index) const;
};
```

4. C1層インターフェース（taro担当）

4.1 Client

関数	戻り値	呼び出し元	説明
<code>int getFd() const</code>	<code>int</code>	B, C2	fdを取得
<code>const std::string& getNick() const</code>	<code>const std::string&</code>	B, C2	nickを取得
<code>const std::string& getUsername() const</code>	<code>const std::string&</code>	B	usernameを取得
<code>const std::string& getRealname() const</code>	<code>const std::string&</code>	B	realnameを取得
<code>const std::string& getHost() const</code>	<code>const std::string&</code>	B	hostを取得
<code>std::string getFullPrefix() const</code>	<code>std::string</code>	B	<code>nick!user@host</code> を返す
<code>void setUsername(const std::string&)</code>	<code>void</code>	B	username設定
<code>void setRealname(const std::string&)</code>	<code>void</code>	B	realname設定
<code>void setHost(const std::string&)</code>	<code>void</code>	A	host設定（接続時）
<code>void setPassOk(bool)</code>	<code>void</code>	B	PASS成功状態設定
<code>bool isPassOk() const</code>	<code>bool</code>	B	PASS済みか
<code>bool isRegistered() const</code>	<code>bool</code>	B	登録完了か
<code>bool canRegister() const</code>	<code>bool</code>	B	登録可能か （PASS/NICK/USER揃った）
<code>void markRegistered()</code>	<code>void</code>	B	登録完了にする

⚠ **注意:** `setNick()` は外部から直接呼ばない。 `ServerState::updateNick()` を使う。

`getFullPrefix()` は RFC 1459 Section 2.3 に基づき、サーバー→クライアントのメッセージに必要。詳細は `rfc1459_prefix_analysis.md` 参照。

4.2 ServerState

関数	戻り値	呼び出し元	説明
<code>const std::string& password() const</code>	<code>const std::string&</code>	B	サーバパスワード
<code>void addClient(int fd)</code>	<code>void</code>	Server	Client作成
<code>void removeClient(int fd)</code>	<code>void</code>	Server, B	Client削除 (Channel参照も除去)
<code>Client* getClientByFd(int fd)</code>	<code>Client*</code>	B	fdからClient取得
<code>Client* getClientByNick(const std::string&)</code>	<code>Client*</code>	B	nickからClient取得
<code>bool nickExists(const std::string&) const</code>	<code>bool</code>	B	nick重複確認
<code>void updateNick(Client&, const std::string&)</code>	<code>void</code>	B	nick変更 (辞書も更新)
<code>Channel* getChannel(const std::string&)</code>	<code>Channel*</code>	B	Channel取得 (なければNULL)
<code>Channel* getOrCreateChannel(const std::string&)</code>	<code>Channel*</code>	B	Channel取得or作成
<code>void removeChannelIfEmpty(const std::string&)</code>	<code>void</code>	B	空Channel削除

5. C2層インターフェース (hanako担当)

5.1 Channel

関数	戻り値	呼び出し元	説明
<code>const std::string& name() const</code>	<code>const std::string&</code>	B, C1	channel名
<code>bool hasMember(Client*) const</code>	<code>bool</code>	B	参加済みか
<code>void addMember(Client*)</code>	<code>void</code>	B	member追加
<code>void removeMember(Client*)</code>	<code>void</code>	B	member削除
<code>void removeClient(Client*)</code>	<code>void</code>	B, C1	member/operator/invited一括削除
<code>std::vector<Client*> members() const</code>	<code>std::vector<Client*></code>	B	member一覧
<code>size_t memberCount() const</code>	<code>size_t</code>	B	member数
<code>bool isOperator(Client*) const</code>	<code>bool</code>	B	operator権限確認
<code>void addOperator(Client*)</code>	<code>void</code>	B	operator付与
<code>void removeOperator(Client*)</code>	<code>void</code>	B	operator剥奪
<code>void invite(Client*)</code>	<code>void</code>	B	招待リスト追加
<code>bool isInvited(Client*) const</code>	<code>bool</code>	B	招待済みか

関数	戻り値	呼び出し元	説明
<code>void removeInvite(Client*)</code>	<code>void</code>	B	招待解除
<code>void setTopic(const std::string&)</code>	<code>void</code>	B	topic設定
<code>const std::string& topic() const</code>	<code>const std::string&</code>	B	topic取得
<code>ChannelModes& modes()</code>	<code>ChannelModes&</code>	B	mode変更用
<code>const ChannelModes& modes() const</code>	<code>const ChannelModes&</code>	B	mode参照用
<code>bool isEmpty() const</code>	<code>bool</code>	B, C1	member 0人か

5.2 ChannelModes

関数	戻り値	呼び出し元	説明
<code>bool inviteOnly() const</code>	<code>bool</code>	B	+i状態
<code>bool topicRestricted() const</code>	<code>bool</code>	B	+t状態
<code>bool hasKey() const</code>	<code>bool</code>	B	+k状態
<code>const std::string& key() const</code>	<code>const std::string&</code>	B	パスワード
<code>int limit() const</code>	<code>int</code>	B	人数制限 (-1=無制限)
<code>void setInviteOnly(bool)</code>	<code>void</code>	B	+i/-i設定
<code>void setTopicRestricted(bool)</code>	<code>void</code>	B	+t/-t設定
<code>void setKey(const std::string&)</code>	<code>void</code>	B	+k設定
<code>void unsetKey()</code>	<code>void</code>	B	-k設定
<code>void setLimit(int)</code>	<code>void</code>	B	+l設定
<code>void unsetLimit()</code>	<code>void</code>	B	-l設定

6. 重要ルール

6.1 【実装】nick変更は ServerState 経由

```
// ❌ NG
client.setNick("newNick");

// ✅ OK
state.updateNick(client, "newNick");
```

6.2 【設計】Client削除は ServerState 経由

```
// ServerState::removeClient(fd) が自動処理:
// - 全Channelからmember/operator/invited削除
// - fd辞書・nick辞書の更新
// - 空Channelの削除
```

6.3 【設計】operator権限は Channel が管理

```
// ❌ NG
client.setOperator(true);

// ✅ OK
channel.addOperator(&client);
```

6.4 【設計】Channel は Client を所有しない

- Channel は Client* を保持するだけ
- delete は ServerState の責務

6.5 【設計】B層は送信処理を行わない

- CommandDispatcher は send() を呼ばない
- 結果は CommandResult に詰めて返す
- A層が CommandResult を適用して送信

7. 未確定事項（Open Questions）

項目	現状	決定タイミング
Poller分離	optional	実装時判断
ConnectionManager分離	optional	実装時判断
ClientRegistry分離	optional	実装時判断
ChannelService分離	optional	実装時判断

8. 関連ドキュメント

ドキュメント	内容
design.md	全体設計・責務分割
ref_interface.md	A/B層の詳細インターフェース（リファレンス用）
onboarding_A.md	A層オンボーディング
onboarding_B.md	B層オンボーディング
onboarding_C1.md	taro向けオンボーディング
onboarding_C2.md	hanako向けオンボーディング
rfc1459_prefix_analysis.md	getFullPrefix() の根拠